Lecture Notes for

# Neural Networks I

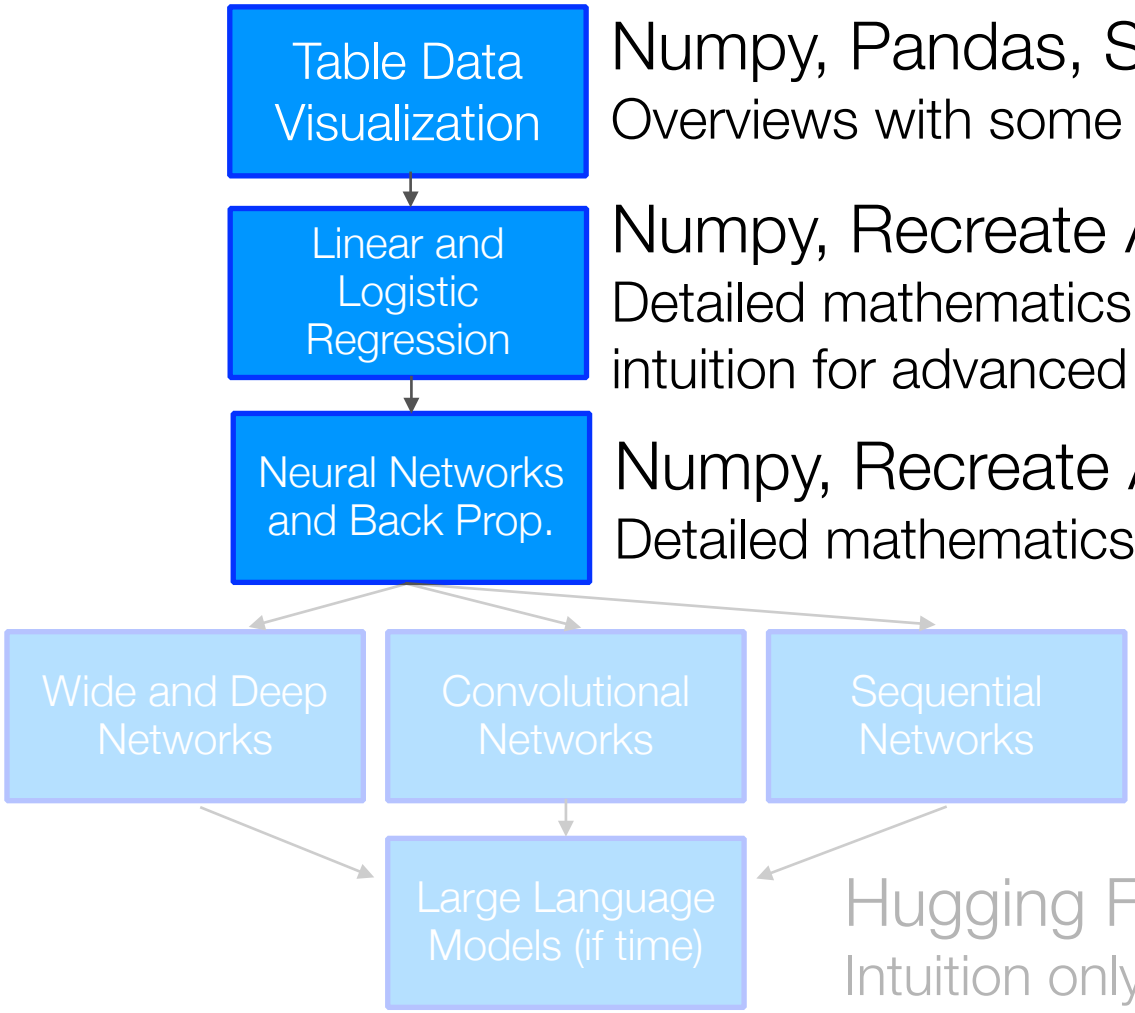



## Optimizing Neural Networks

# Class Logistics and Agenda

- Logistics
  - Grading
- Agenda:
  - Last time: Flipped Module on Back-propagation
  - Practical Multi-layer Architectures
    - API Basics, Auto-differentiation
    - Momentum and Learning rates
- Next Time: More MLPs
    - Objective Functions
    - Weight Initialization
    - More activations: Tanh, ReLU, SiLU
    - Adaptive learning: AdaGrad, AdaM, etc.

# Class Overview, by topic




Numpy, Pandas, Seaborn
Overviews with some in-depth discussion

Numpy, Recreate API for Scikit-learn
Detailed mathematics for simple optimization
intuition for advanced optimization

Numpy, Recreate API for DL Frameworks
Detailed mathematics for NN operations

Hugging Face
Intuition only

# Semester Summary, so far!

Adaline Network
*Widrow and Hoff, 1960*

Same as linear regression

$$J(\mathbf{w}) = \sum_{i=1}^{M} (y^{(i)} - \mathbf{w} \cdot \mathbf{x}^{(i)})^2 \qquad \mathbf{w} = (\mathbf{X}^T \cdot \mathbf{X})^{-1} \cdot \mathbf{X}^T \cdot y$$



Perceptron Binary
*Frank Rosenblatt, 1960's*

Same as binary logistic regression

$$\phi(\mathbf{w} \cdot \mathbf{x} + \mathbf{b}) = \frac{1}{1 + \exp(-\mathbf{w} \cdot \mathbf{x} + \mathbf{b})}$$

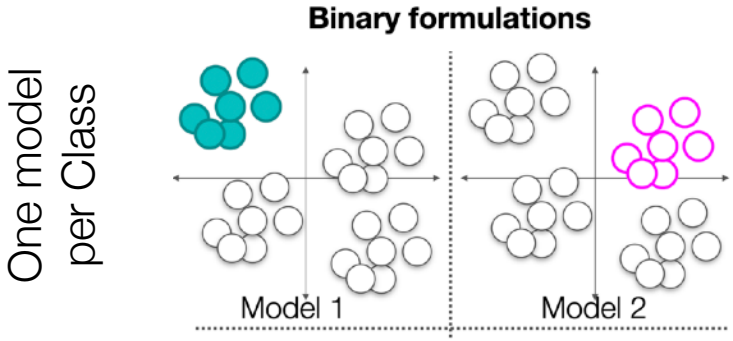
$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \text{mean}\left((\mathbf{y} - g(\mathbf{X} \cdot \mathbf{w})) \odot \mathbf{X}\right)_{cols}$$

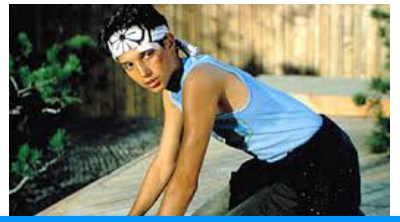


Perceptron Multi-class
*Frank Rosenblatt, 1960's*

Same as multi-class logistic regression

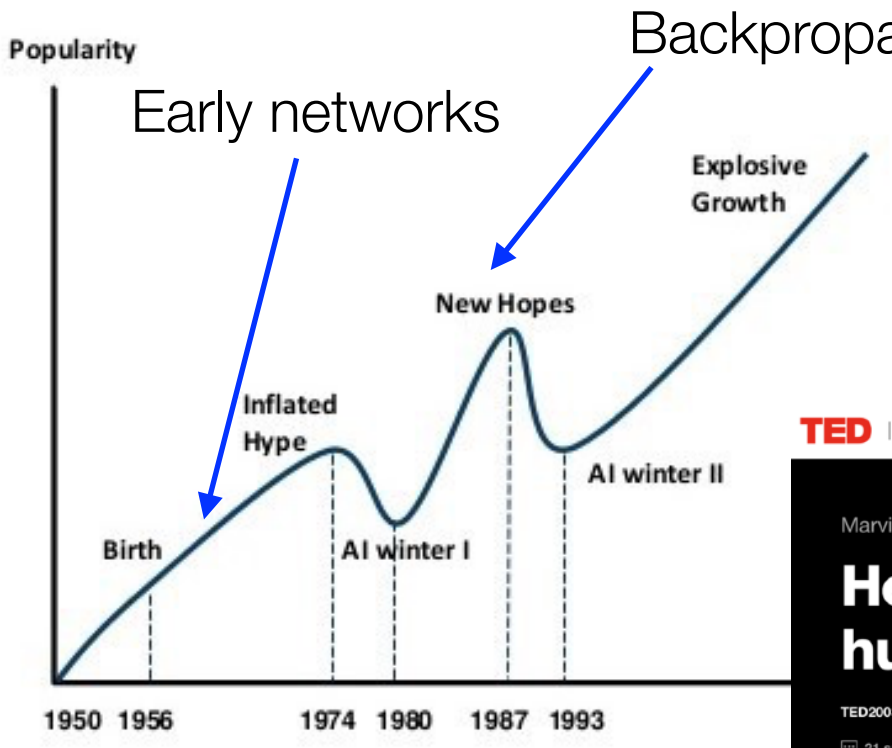
*one-versus-all*



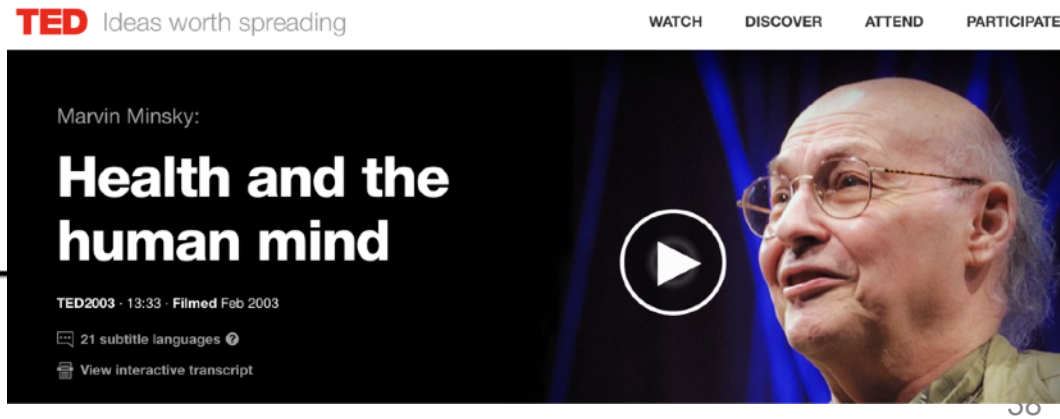
these networks were created in the 50's and 60's but were abandoned **why were they not used?**

# The First AI Winter (if not covered already)









# The Rosenblatt-Widrow-Hoff Dilemma

- 1960's: Rosenblatt got into a public academic argument with Marvin Minsky and Seymour Papert

> "Given an elementary α-perceptron, a stimulus world W, and any classification C(W) for which a solution exists; let all stimuli in W occur in any sequence, provided that each stimulus must reoccur in finite time; then beginning from an arbitrary initial state, an error correction procedure will always yield a solution to C(W) in finite time…"

- Minsky and Papert publish limitations paper, 1969:

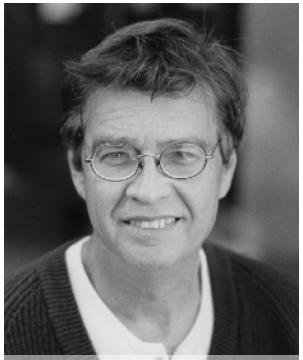
> "the style of research being done on the perceptron is doomed to failure because of these limitations."

- Widrow and Rosenblatt try to build bigger networks without limitations and fail
  - ⋆ Neural Networks research **basically stops** for **17 years**
- **Until**: researchers revisit training bigger networks
  - ⋆ Neural Networks with multiple layers

# Stable Training of Multi-layer Architectures: history

- 1986: *Rumelhart*, *Hinton*, and *Williams* popularize gradient calculation for multi-layer network
  - *technically* introduced by Werbos in ~1982
- **difference**: Rumelhart *et al.* validated ideas with a computer
- until this point no one could train a multiple layer network consistently
- algorithm is popularly called **Back-Propagation**
- wins pattern recognition prize in 1993, becomes de-facto machine learning algorithm until: SVMs and Random Forests in ~2004
- would eventually see a resurgence for its ability to train algorithms for Deep Learning applications: **Hinton is widely considered the founder of deep learning**

David Rumelhart



1942-2011

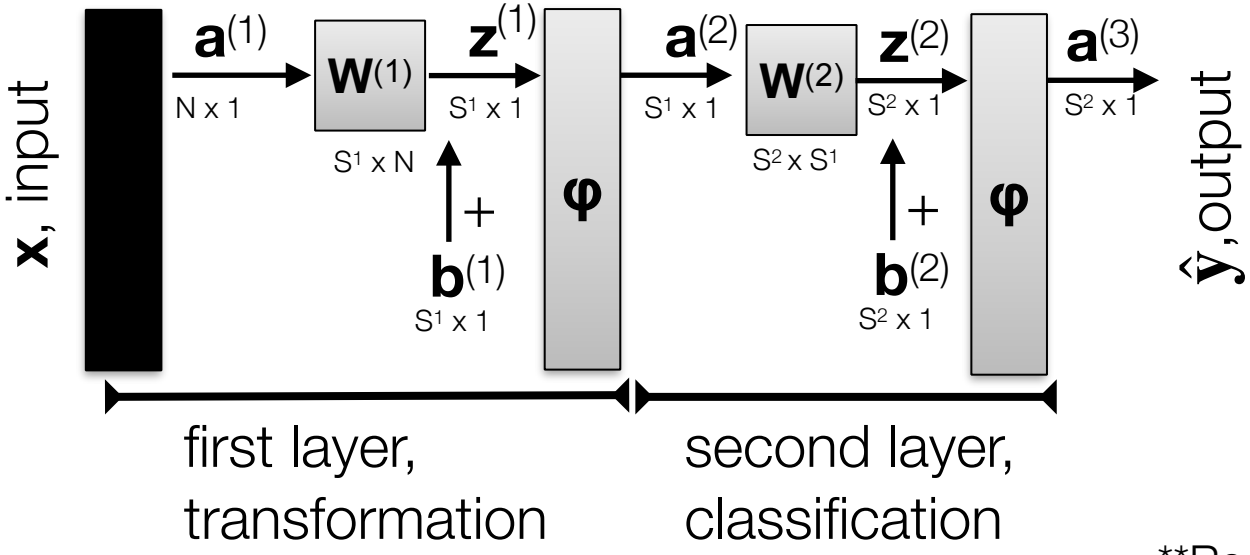
Geoffrey Hinton



# Review: Back propagation

- Optimize all weights of network at once
- Steps:
  1. Forward propagate to get all $\mathbf{Z}^{(l)}$ , $\mathbf{A}^{(l)}$
  2. Get final layer gradient
  3. Back propagate sensitivities
  4. Update each $\mathbf{W}^{(l)}$



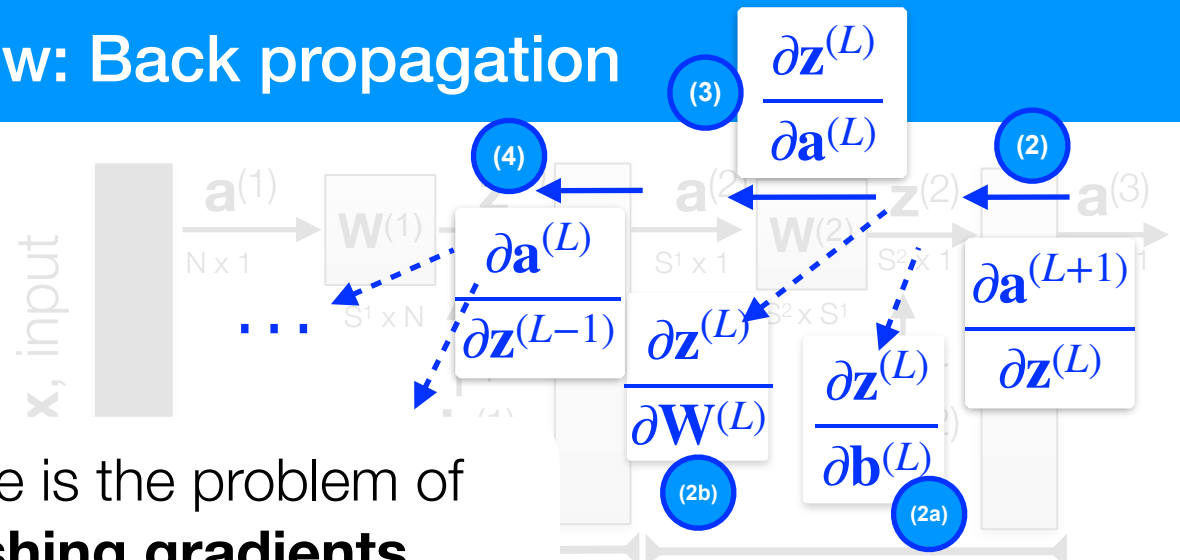

$$J(\mathbf{W}) = \left\| \mathbf{Y} - \hat{\mathbf{Y}} \right\|^2$$

$$w_{i,j}^{(l)} \leftarrow w_{i,j}^{(l)} - \eta \frac{\partial J(\mathbf{W})}{\partial w_{i,j}^{(l)}}$$

$$b_i^{(l)} \leftarrow b_i^{(l)} - \eta \frac{\partial J(\mathbf{W})}{\partial b_i^{(l)}}$$

**Recall from Flipped Assignment!

# Review: Back propagation




$$J(\mathbf{W}) = \left\| \mathbf{Y} - \hat{\mathbf{Y}} \right\|^2$$

$$w_{i,j}^{(l)} \leftarrow w_{i,j}^{(l)} - \eta \frac{\partial J(\mathbf{W})}{\partial w_{i,j}^{(l)}}$$

$$b_i^{(l)} \leftarrow b_i^{(l)} - \eta \frac{\partial J(\mathbf{W})}{\partial b_i^{(l)}}$$

Where is the problem of **vanishing gradients** introduced?

**Recall from Flipped Assignment!

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial J(\cdot)}{\partial \mathbf{b}^{(l)}}$$

$$\frac{\partial J(\cdot)}{\partial \mathbf{b}^{(L)}} = \frac{\partial \mathbf{z}^{(L)}}{\partial \mathbf{b}^{(L)}} \cdot \frac{\partial \mathbf{a}^{(L+1)}}{\partial \mathbf{z}^{(L)}} \cdot \frac{\partial J(\cdot)}{\partial \mathbf{a}^{(L+1)}}$$

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial J(\cdot)}{\partial \mathbf{W}^{(l)}}$$

$$\frac{\partial J(\cdot)}{\partial \mathbf{W}^{(L)}} = \frac{\partial \mathbf{z}^{(L)}}{\partial \mathbf{W}^{(L)}} \cdot \frac{\partial \mathbf{a}^{(L+1)}}{\partial \mathbf{z}^{(L)}} \cdot \frac{\partial J(\cdot)}{\partial \mathbf{a}^{(L+1)}}$$

chain rule for parameter updates

$$\frac{\partial J(\cdot)}{\partial \mathbf{W}^{(L-1)}} = \frac{\partial \mathbf{z}^{(L-1)}}{\partial \mathbf{W}^{(L-1)}} \cdot \frac{\partial \mathbf{a}^{(L)}}{\partial \mathbf{z}^{(L-1)}} \cdot \frac{\partial \mathbf{z}^{(L)}}{\partial \mathbf{a}^{(L)}} \cdot \frac{\partial \mathbf{a}^{(L+1)}}{\partial \mathbf{z}^{(L)}} \cdot \frac{\partial J(\cdot)}{\partial \mathbf{a}^{(L+1)}}$$

(4b) (4) (3) (2) (1)

# 06. MLP Neural Networks API.ipynb

Demo



## SciKit-Learn:

```
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(
    hidden_layer_sizes=(64, 32),  activation='sigmoid',
    solver='sgd', max_iter=1000 )

mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)
```

Fit / Predict on classifier object

## Modern APIs:

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(feat_dim, 64), nn.Sigmoid(),
    nn.Linear(64, 32),       nn.Sigmoid(),
    nn.Linear(32, 1) ,       nn.Sigmoid() )

optimizer = SGD( model.parameters(), lr=0.001)

for X_train, y_train in loader():
    optimizer.zero_grad()

    outputs = model(X_train)
    loss = MSELoss(outputs, y_train)

    loss.backward()
    optimizer.step()
```

Build model as a list of operations

optimizer selection via object

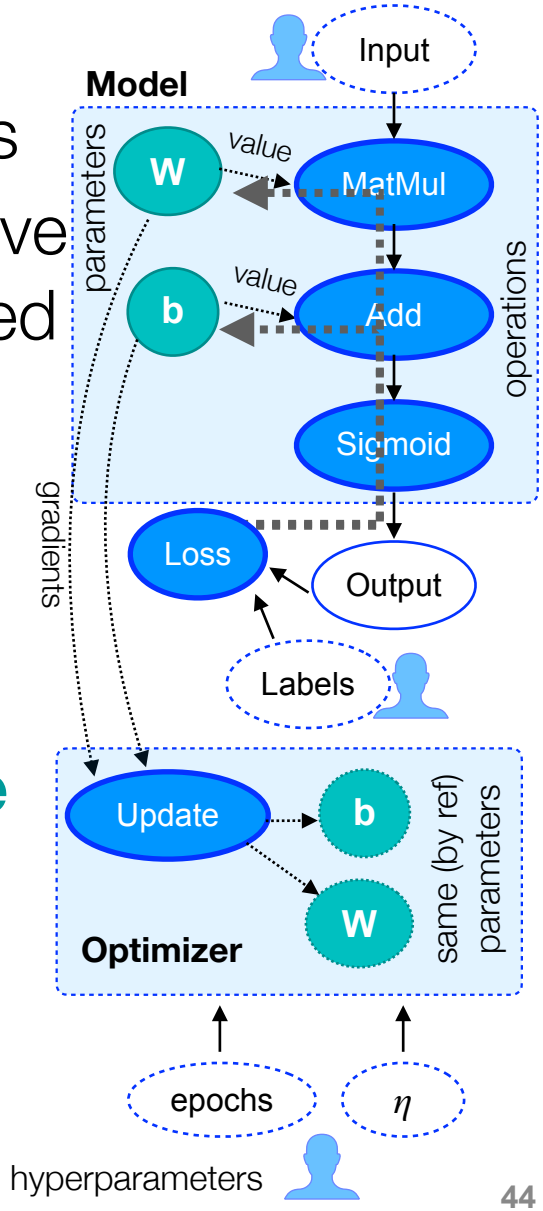
place data in model (feedforward)

calculate objective function

get gradients and update with optimizer

# Building a Neural Nets API

- Flipped module mirrored `sklearn`, but we want to emulate modern deep learning APIs
- Want to build up a set of operations that have **`forward`** and **`backward`** operations defined by user
- We will define a **`parameters`** class such that they store their `value` and `gradient`
  - All **`parameters`** will be `Numpy` arrays, allowing us to **pass them by reference** across classes
- Need a **`model`** class that can place these operations/parameters together and apply chain rule to get gradients



# Building a Neural Nets API

```
class Parameter:
    def __init__(self, value):
        self.value = value
        self.grad = np.zeros_like(value)
```

W b

`Parameters` store `value` and `grad`

```
class Layer:
    def __init__(self):
        self.input = None
        self.output = None
    def __call__(self, input): pass
    def backward(self, sensitivity_in): pass
    def parameters(self): return []
    def set_parameters(self, parameters): pass
    def zero_grad(self): pass
```

MatMul

```
layer(features)
layer.backward(sensitivity)
```

`Layer` defines forward / backward
(and use sensitivity as input to backward)

```
class Loss:
    def __init__(self): pass
    def __call__(self, y_true, y_pred): pass
    def backward(self): pass
```

Loss

`Loss` must provide a way to calculate gradient, for starting back prop

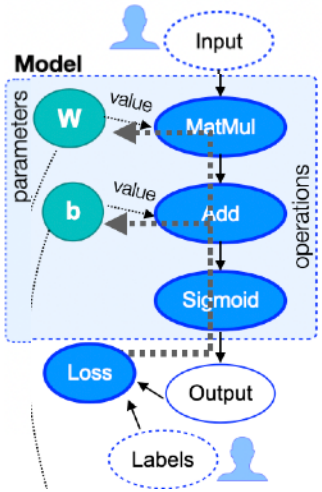
```
class Model:
    def __init__(self, layers):
        self.layers = layers
        self.parameters = []
        for layer in self.layers:
            self.parameters.extend(layer.parameters())

    def __call__(self, input):
        output = input
        for layer in self.layers:
            output = layer(output)
        return output

    def backward(self, loss_sensitivity):
        sensitivity = loss_sensitivity
        for layer in reversed(self.layers):
            sensitivity = layer.backward(sensitivity)
```

Model saves a list of parameters (by reference)

Model `__call__` applies operations




Back prop updates parameter gradients

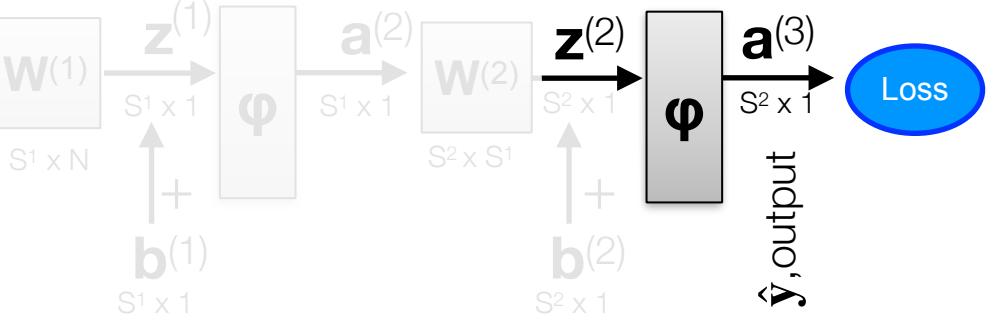
# Loss and Sigmoid in our Neural Nets API

```
class MSELoss(Loss):
    def __call__(self, y_true, y_pred):
        self.y_true = y_true
        self.y_pred = y_pred
        self.output = np.mean(np.power(y_true - y_pred, 2))
        return self.output

    def backward(self):
        return 2 * (self.y_pred - self.y_true) / self.y_true.shape[1]
```

Loss value

Loss gradient




$$\frac{\partial J(\mathbf{W})}{\partial \mathbf{z}^{(L)}} = \frac{\partial \mathbf{a}^{(L+1)}}{\partial \mathbf{z}^{(L)}} \frac{\partial J(\mathbf{W})}{\partial \mathbf{a}^{(L+1)}}$$

```
class Sigmoid(Layer):

    def __call__(self,input):
        self.input = input
        self.output = expit(input)
        return self.output

    def backward(self, sensitivity_in):
        a = self.output
        return a * (1 - a) * sensitivity_in
```

$$\frac{\partial J(\mathbf{W})}{\partial [\mathbf{z}^{(2)}]^{(k)}} = \underbrace{[\mathbf{a}^{(3)}]^{(k)} * \left(1 - [\mathbf{a}^{(3)}]^{(k)}\right)}_{\text{Sigmoid Derivative}} * \underbrace{\frac{2}{M}\left([\mathbf{a}^{(3)}]^{(k)} - \mathbf{y}^{(k)}\right)}_{\text{MSE derivative}}$$

```
model.backward( loss.backward() )
```

# Common Layer in Neural Nets API

setup

```
class Dense(Layer):
    def __init__(self, input_size, output_size):
        self.weights = Parameter(np.random.uniform(-1.0, 1.0,
                                 size=(output_size, input_size)) )
        self.bias = Parameter(np.zeros((output_size, 1)))
        super().__init__()

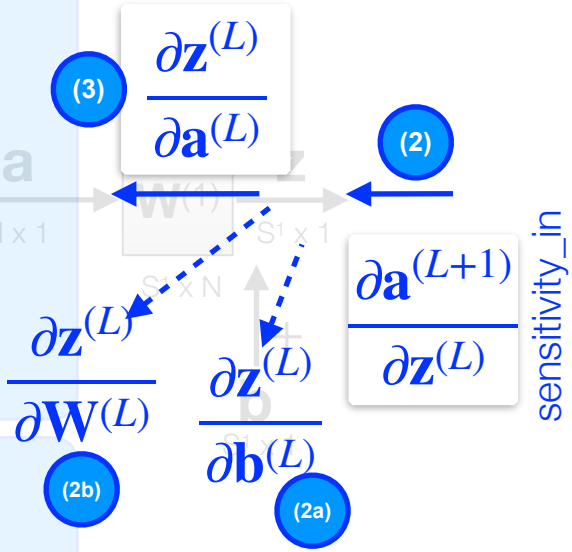
    def zero_grad(self):
        self.weights.grad.fill(0)
        self.bias.grad.fill(0)

    def parameters(self):
        # these are passing the numpy references back
        return [self.weights, self.bias]
```

forward

```
def __call__(self, input):
    self.a = input
    self.z = self.weights.value @ self.a + self.bias.value
    return self.z
```

$$\mathbf{z}^{(L)} = \mathbf{W}^{(L)} \cdot \mathbf{a}^{(L)} + \mathbf{b}^{(L)}$$




backward

```
def backward(self, sensitivity_in):
    self.weights.grad += sensitivity_in @ self.a.T
    self.bias.grad += np.sum(sensitivity_in, axis=1, … )

    return self.weights.value.T @ sensitivity_in
```

save update params

$$\frac{\partial \mathbf{z}^{(l)}}{\partial w_{i,j}^{(l)}} = a_{j,i}$$

from flipped module

$$\frac{\partial \mathbf{z}^{(l)}}{\partial a_{i,j}} = w_{j,i}^{(l)}$$

from flipped module

return new sensitivity

# Training with Neural Nets API

```
def train_step(model, loss, x_train, y_train, learning_rate = 0.1):

    model.zero_grad()
    output = model(x_train.T)
    error  = loss(y_train.T, output)
    model.backward(loss.backward())

    for param in model.parameters:
        param.value -= param.grad * learning_rate
```

feedforward and save gradients

gradient descent updates

```
model = Model([Dense(64,32), Sigmoid(),
               Dense(32,10), Sigmoid()])

mse_loss = MSELoss()
```

add operations to model

define loss

```
for epoch in range(500):
    # send model to perform a training step
    train_step(model, mse_loss, X_train, y_train_ohe )
```

```
500/500, error=9.329638317990126e-065
Model: 4 operations.

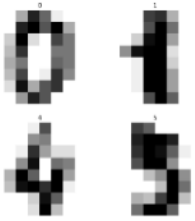
Dense_0          | 2080 Trainable parameters
Sigmoid_1        | 0 Trainable parameters
Dense_2          | 330 Trainable parameters
Sigmoid_3        | 0 Trainable parameters

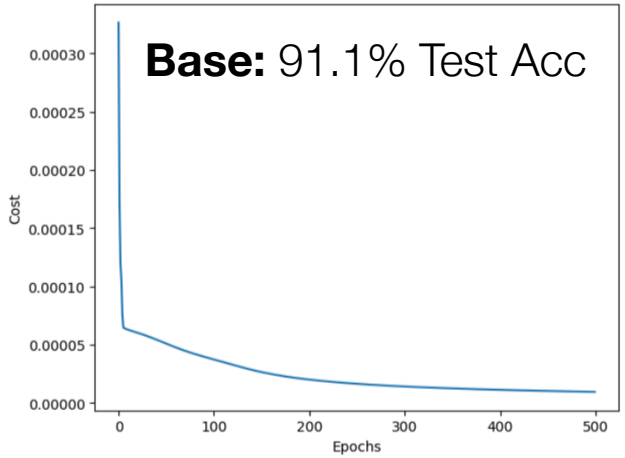
Accuracy on Test Data: 0.9111
```

X_train: flattened pixels
y_train: one hot encoding, 10 classes

numeric digits

*from flipped module*





# Optimization Heuristics

```
def print_message(num_of_times) {
    for i in range(num_of_times) {
        print("Bython is awesome!");
    }
}

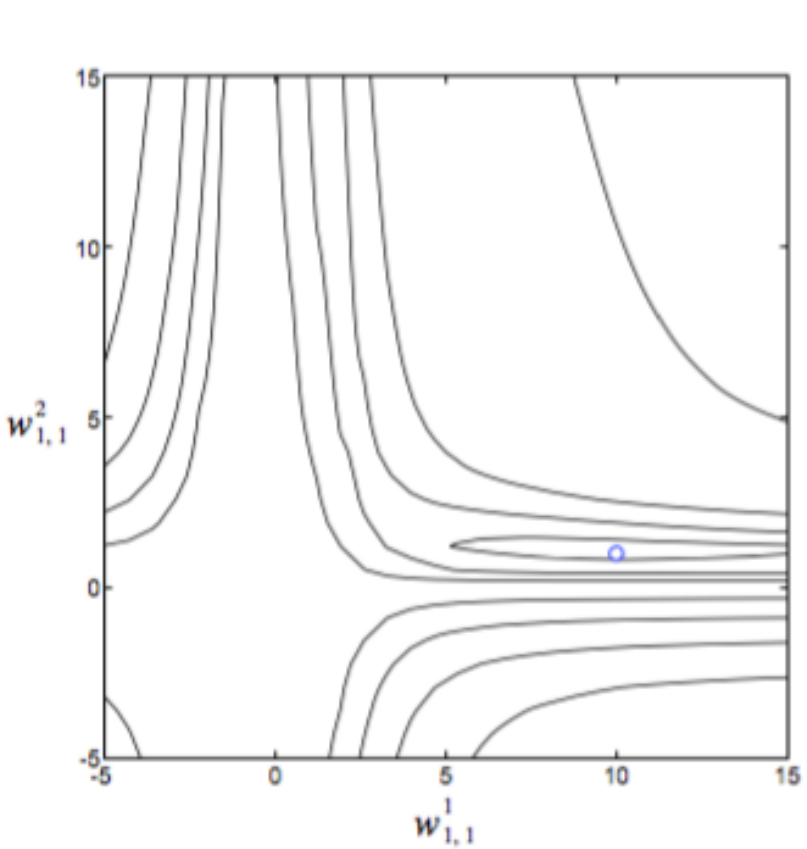
if __name__ == "__main__" {
    print_message(10);
}
```

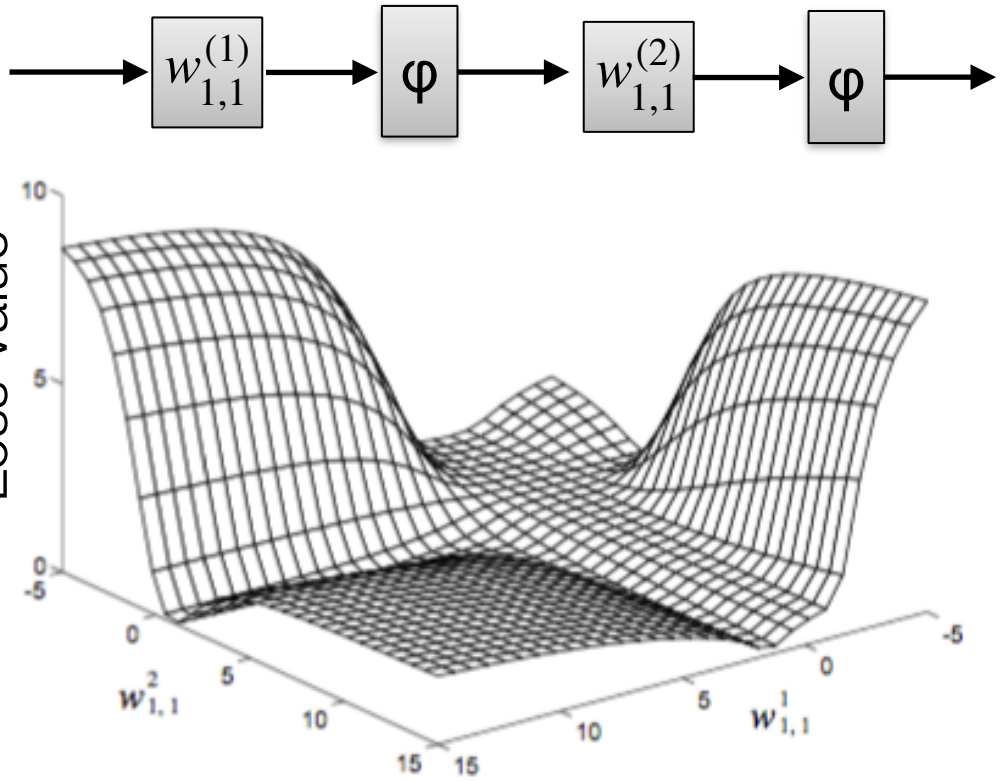
## Bython

Python with braces. Because Python is awesome, but whitespace is awful.

Bython is a Python preprosessor which translates curly brackets into indentation.

# A new Loss landscape



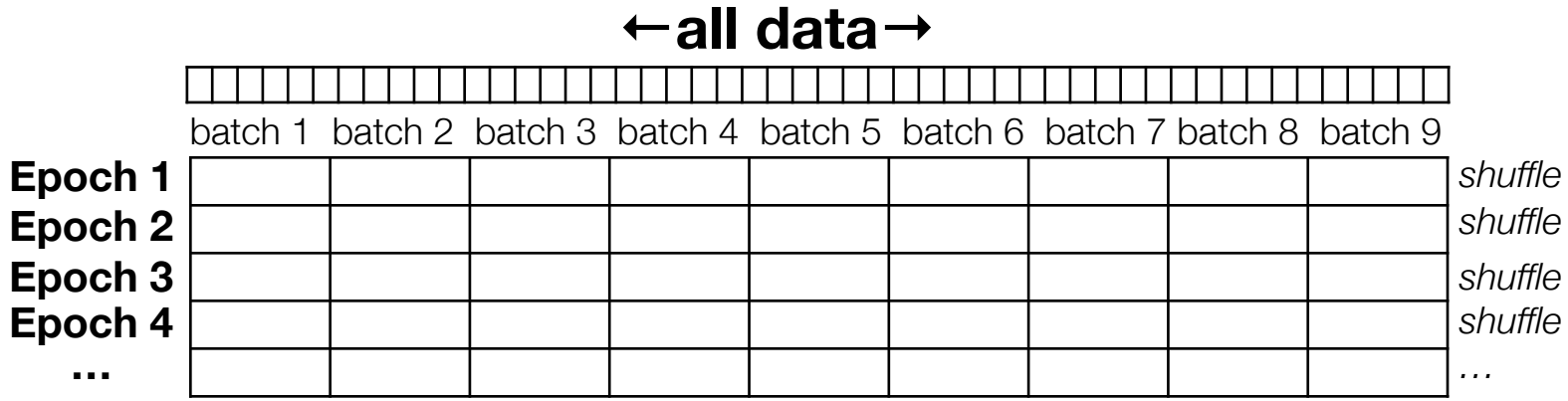




**What are some optimization problems?**

A. There are many local optima that gradients will be fooled by
B. There are many interconnected parameters that change each other
C. There are large flat areas in the loss function, where the gradient is small
D. All of the above

# Same as LR: Mini-batching

- Numerous instances to find one gradient update
  - **solution**: mini-batch



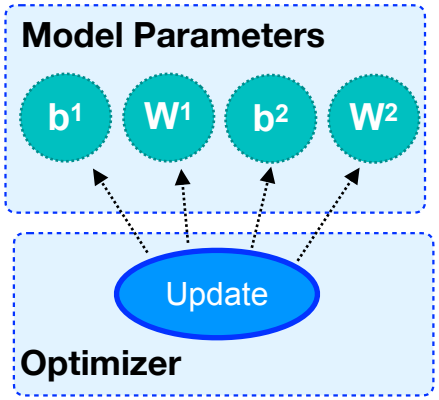

*shuffle ordering **each epoch** and update W's after **each batch***

- **Remaining problems**: there might be many local optima…
  - **solutions**:
    - momentum
    - adaptive learning rate (cooling)

# Adding Dedicated Optimizer, Mini-batching

```python
class SimpleSGD():
    def __init__(self, parameters, learning_rate):
        self.parameters = parameters # passed by numpy reference
        self.learning_rate = learning_rate

    def step(self):
        for param in self.parameters:
            param.value -= param.grad * self.learning_rate
```

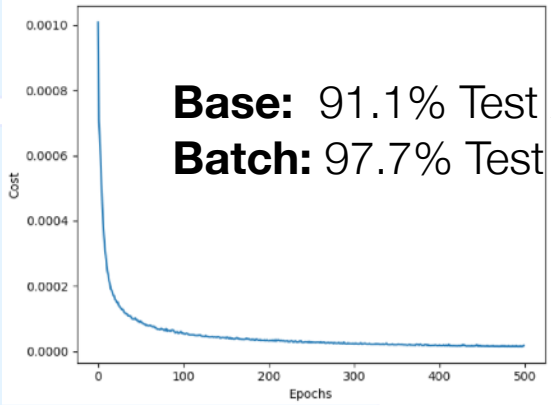



```python
def train_step(model, optimizer, loss, x_train, y_train, batch_size=256):
    indices = np.arange(len(x_train))
    np.random.shuffle(indices)

    error = 0
    for i in range(0, len(indices), batch_size):
        batch_idx = indices[i : i + batch_size]
        model.zero_grad()
        output = model(x_train[batch_idx].T)
        error += loss(y_train[batch_idx].T, output)
        model.backward(loss.backward())
        optimizer.step()
```

```
500/500, error=1.8049449893956653e-05
Accuracy on Test Data: 0.9778
Optimizer for 4 layers with 2410 trainable parameters.
Model: 4 operations.

Dense_0          | 2080 Trainable parameters
Sigmoid_1        | 0 Trainable parameters
Dense_2          | 330 Trainable parameters
Sigmoid_3        | 0 Trainable parameters
```



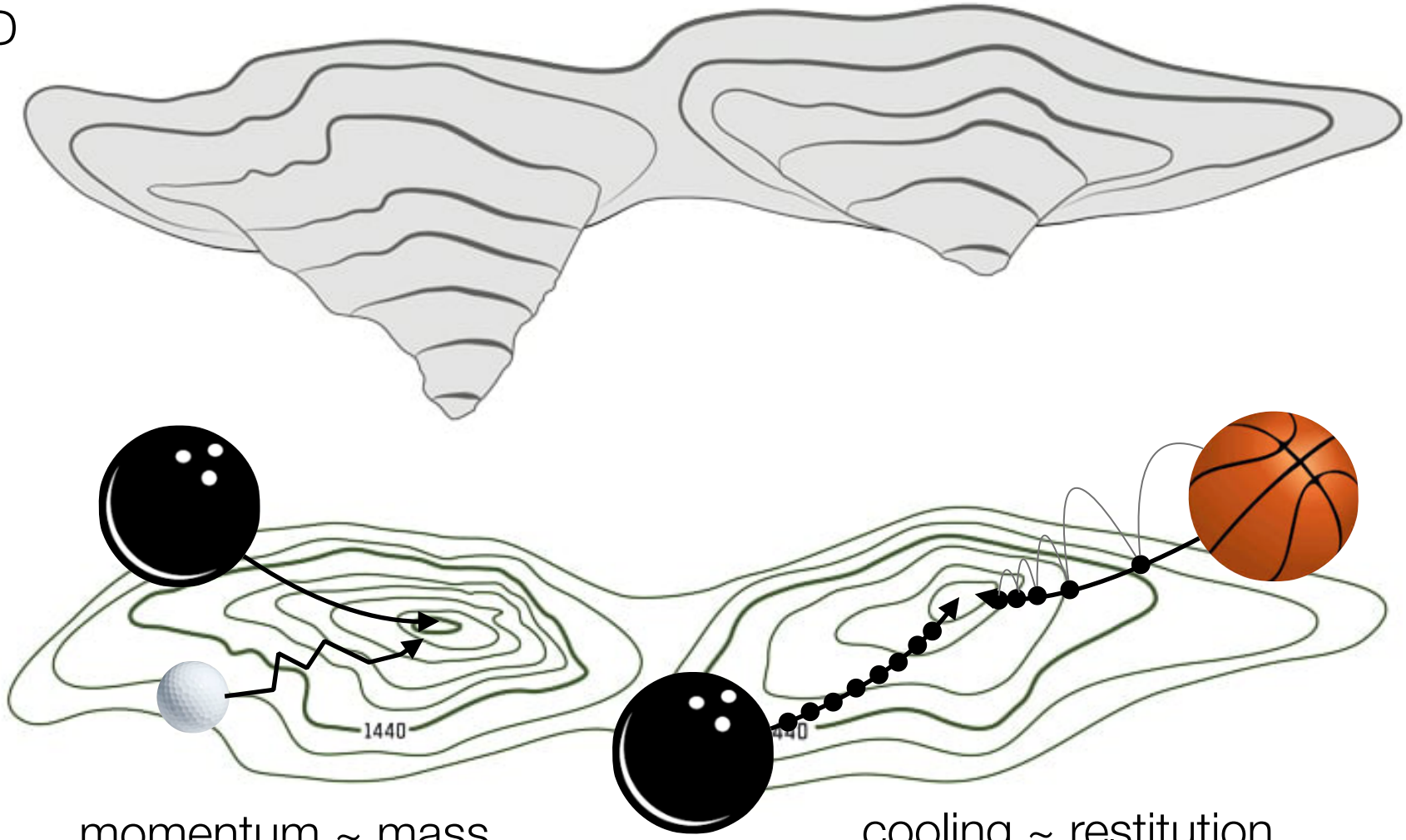

**Base:** 91.1% Test Acc
**Batch:** 97.7% Test Acc

```python
# create model
model = Model([Dense(64,32), Sigmoid(),
               Dense(32,10), Sigmoid()])
# set loss function
mse_loss = MSELoss()
opt = SimpleSGD(model.parameters, learning_rate=1.0)

for epoch in range(500):
    train_step(model, opt, mse_loss, X_train, y_train_ohe,  batch_size=128)
```

# Momentum and Cooling Intuition

3D




Topological

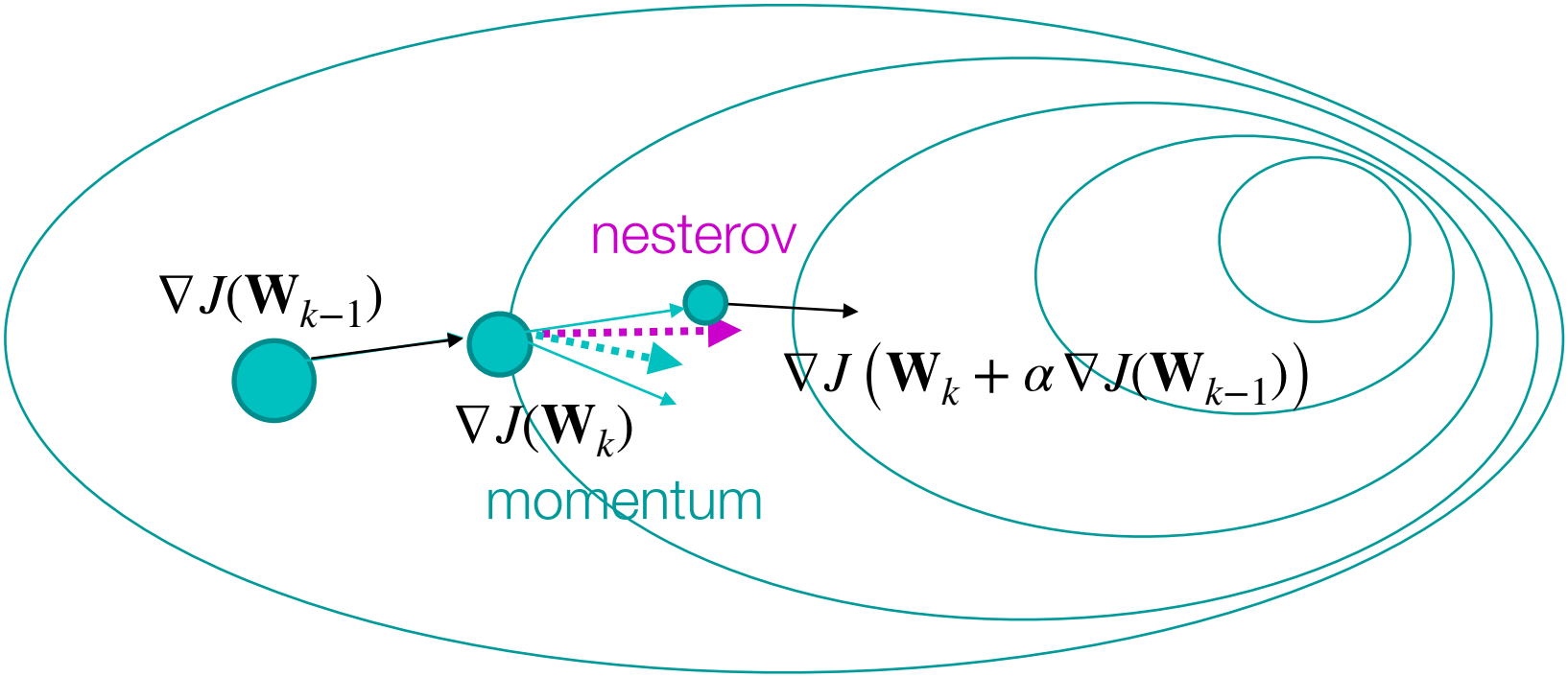
# Momentum

$$\mathbf{W}_{k+1} = \mathbf{W}_k - \rho_k$$

- Momentum $\rho_k = \eta \cdot \nabla J(\mathbf{W}_k) + \alpha \cdot \rho_{k-1}$

- Nesterov's Accelerated Gradient
  *(not as popular as vanilla momentum)*

$$\rho_k = \underbrace{\beta \nabla J\left(\mathbf{W}_k + \alpha \nabla J(\mathbf{W}_{k-1})\right)}_{\text{step twice}} + \alpha \nabla J(\mathbf{W}_{k-1})$$



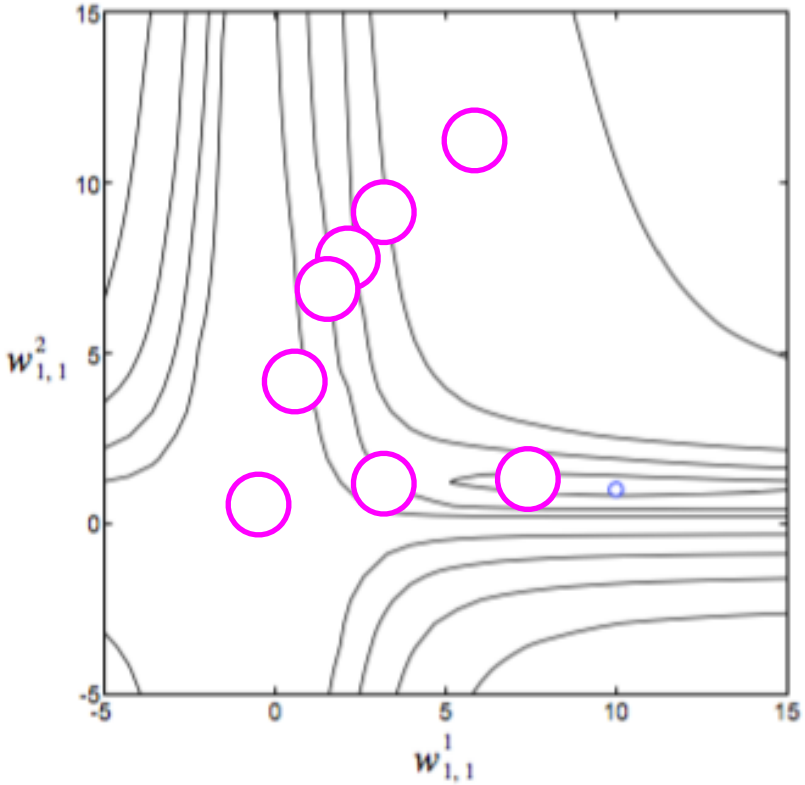
# Cooling (Learning Rate Reduction)

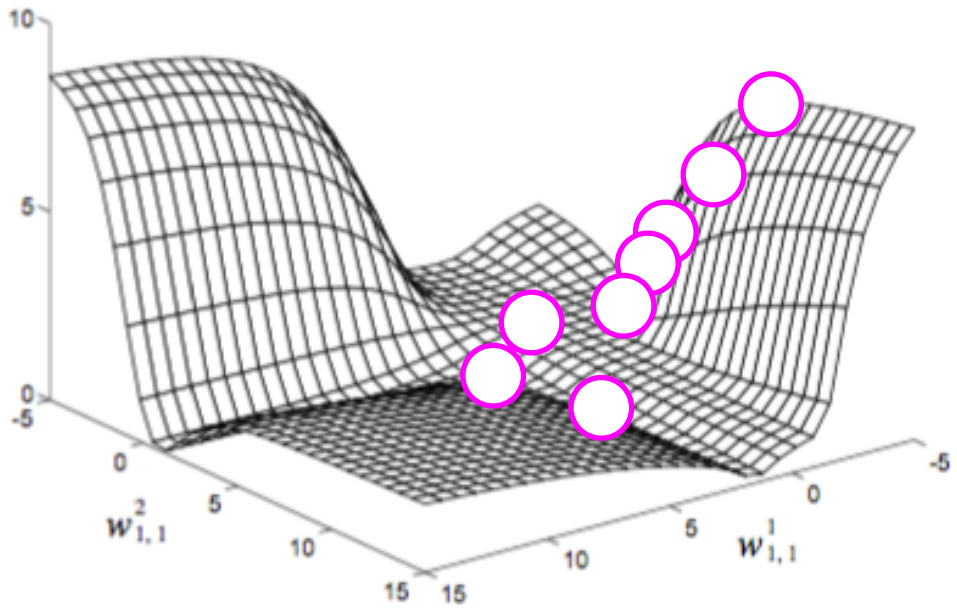
- Fixed Reduction at Each Epoch, $k$

$$\eta_k = \eta_0 \cdot d^{\lfloor \frac{k_{max}}{k} \rfloor}$$ drop by $d$ every $k_d$ epochs

$$\eta_k = \eta_0^{(1+k \cdot d)}$$ drop a little every epoch

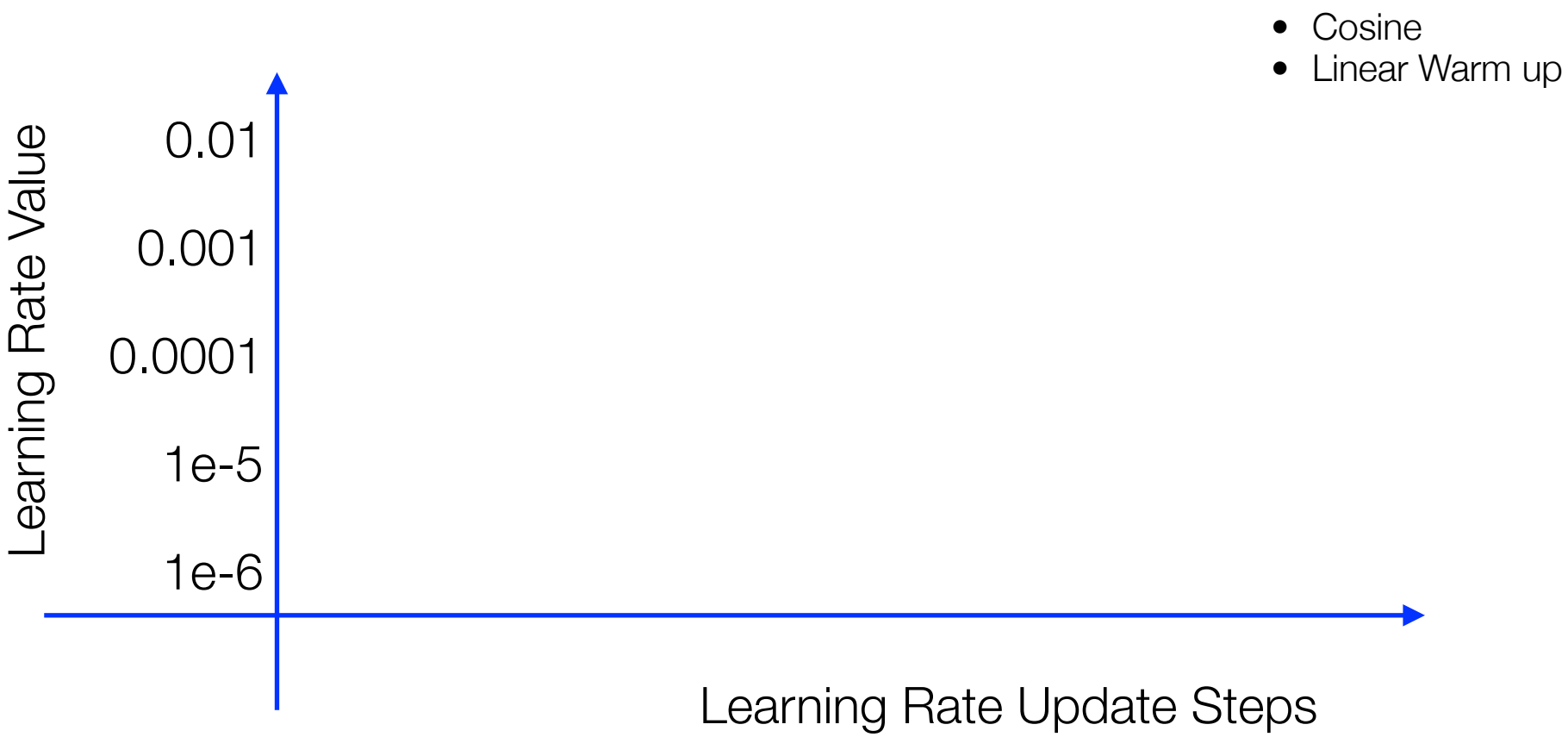
- Adjust on Plateau
  - make smaller when $J$ rapidly changes
  - make bigger when $J$ not changing much



# Learning Rate Schedules

- Many scheduling rate functions exist and can be different for each application
- Some first increase plateau and then decrease



# Adding momentum and cooling

$$\mathbf{W}_{k+1} = \mathbf{W}_k - \eta \cdot \rho_k$$

```
class SGD():
    def __init__(self, parameters, learning_rate, momentum, … ):
        self.parameters = parameters # passed by numpy reference
        self. (… setup parameters … learning_rate, decay_rate, momentum etc.)
        self._cache = [np.zeros_like(p.grad) for p in parameters]

    def step(self):
        for param, cache in zip(self.parameters, self._cache):
            cache[:] = param.grad + self.momentum * cache
            param.value -= self.learning_rate * cache

    def scheduler_step(self, epoch):
        if (epoch % self.step_size) == 0:
            self.learning_rate *= self.decay_rate
```

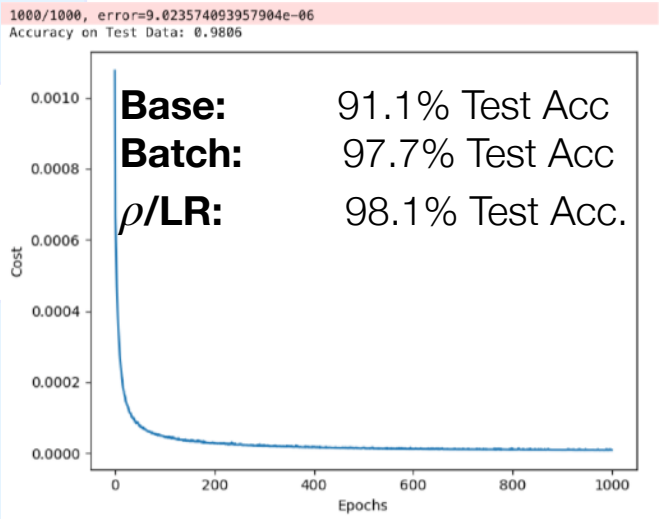
keep numpy c++ reference

Cache for tracking $\rho$ for each parameter

$$\rho_k = \nabla J(\mathbf{W}_k) + \alpha \cdot \rho_{k-1}$$

$$\eta_k = \eta_0 \cdot d^{\lfloor \frac{k_{max}}{k} \rfloor}$$

Need to call `opt.scheduler_step(epoch)` at the end of every epoch



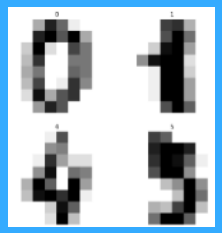

```
# create model
model = Model([Dense(64,32), Sigmoid(),
               Dense(32,10), Sigmoid()])

# set loss function
mse_loss = MSELoss()
opt = SGD(model.parameters, learning_rate=1.0, momentum=0.1, decay_rate=0.9, step_size=100)

for epoch in range(1000):
    train_step(model, opt, mse_loss, X_train, y_train_ohe, batch_size=128)
```

# Summary Thus Far

- Next time is More Optimization Techniques

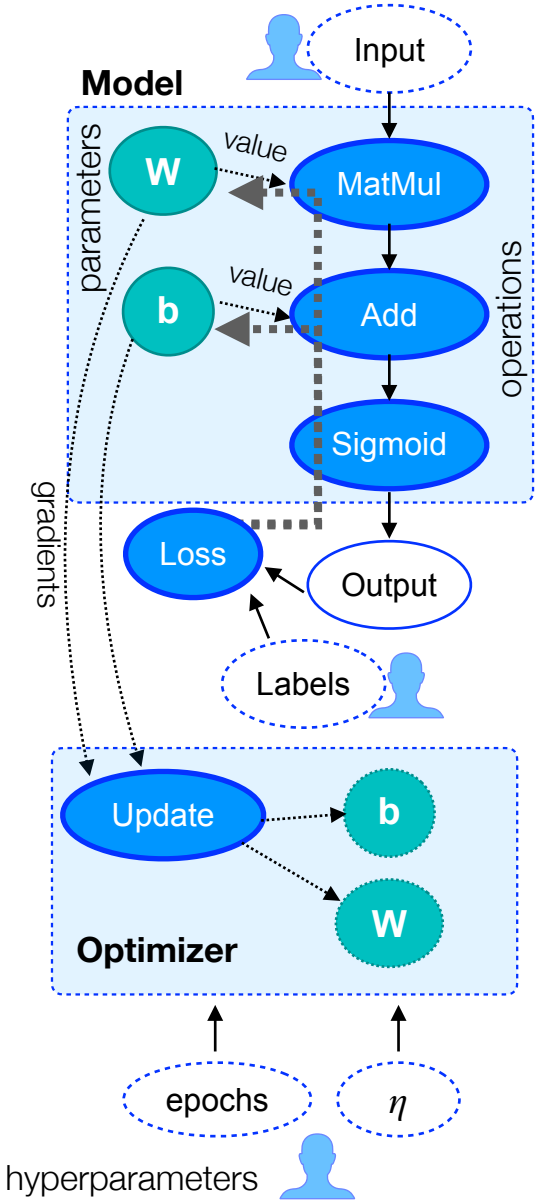


**Digits:** Saturated performance, 98%

New Dataset: **Fashion MNIST**



- larger images
- still 10 classes
- still flatten pixels to make table data
- hard to get more than 85%



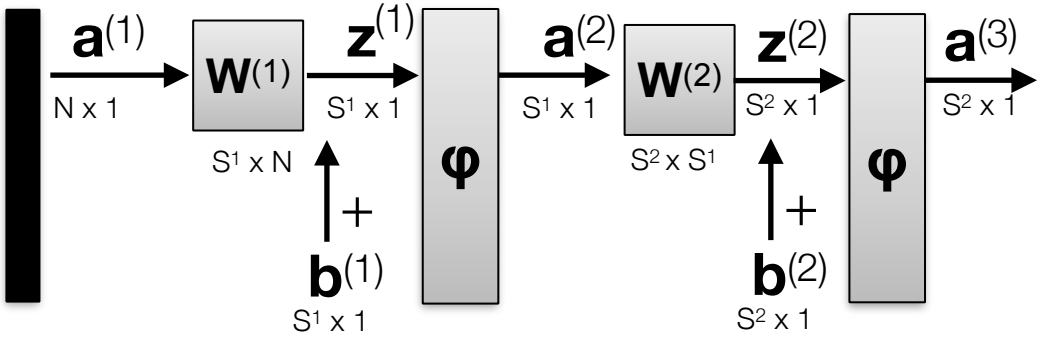

Our API allows us to add many operations that can track their gradients

We can now have full control over loss functions, optimizers, learning rate schedules, etc. via simple sub-classing!

# Objective Function



# Changing the Objective Function




1. Forward propagate to get all $\mathbf{Z}^{(l)}$ , $\mathbf{A}^{(l)}$
2. Get final layer gradient
3. Back propagate sensitivities (chain rule, $\partial \mathbf{z} / \partial \mathbf{a}$)
4. Update each variable (chain rule, $\partial \mathbf{z} / \partial \mathbf{W}$ and $\partial \mathbf{z} / \partial \mathbf{b}$)

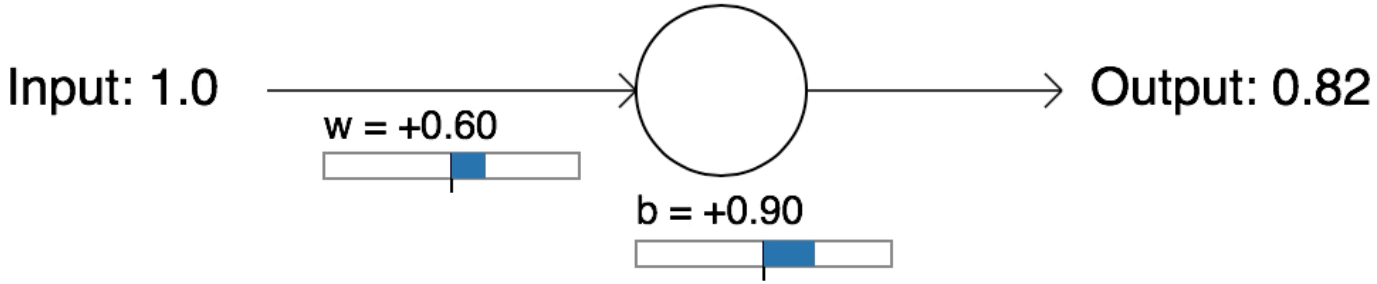
- **Self Test**:
  **True or False**: If we change the cost function, $J(\mathbf{W})$, we only need to update the final layer sensitivity calculation of the back propagation steps. The remainder of the algorithm is unchanged.
  - A. True
  - B. False

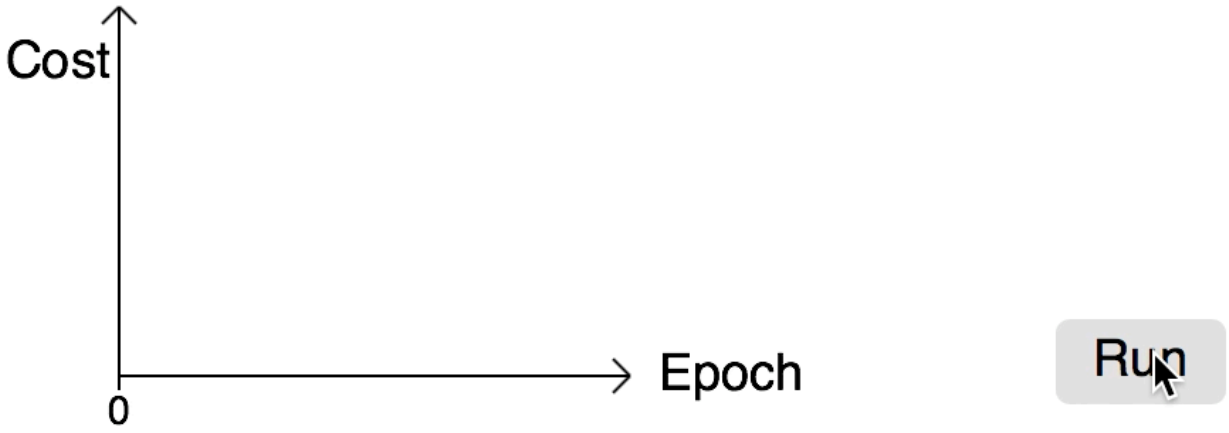
# Practical Implementation of Architectures

- Mean squared error:

$$J(\mathbf{W}) = \sum_{k}^{M} (\mathbf{y}^{(k)} - \mathbf{a}^{(L)})^2$$

least squares objective,
tends to slow training initially



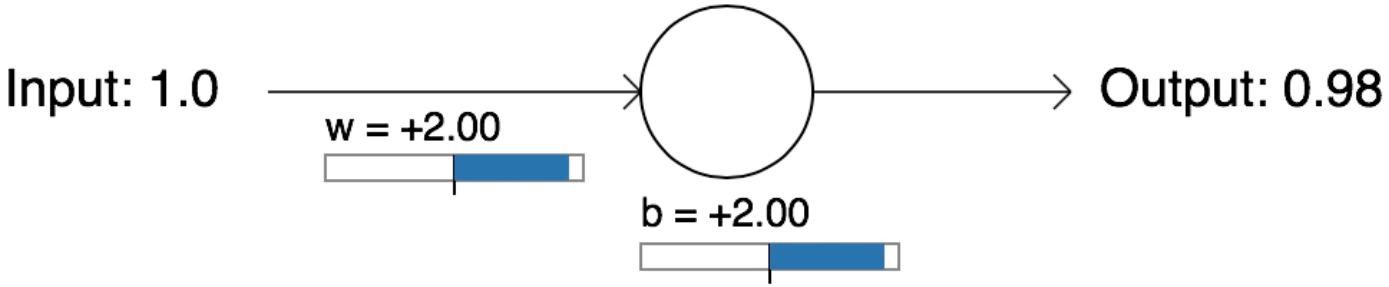

*Neural Networks and Deep Learning*, Michael Nielson, 2015

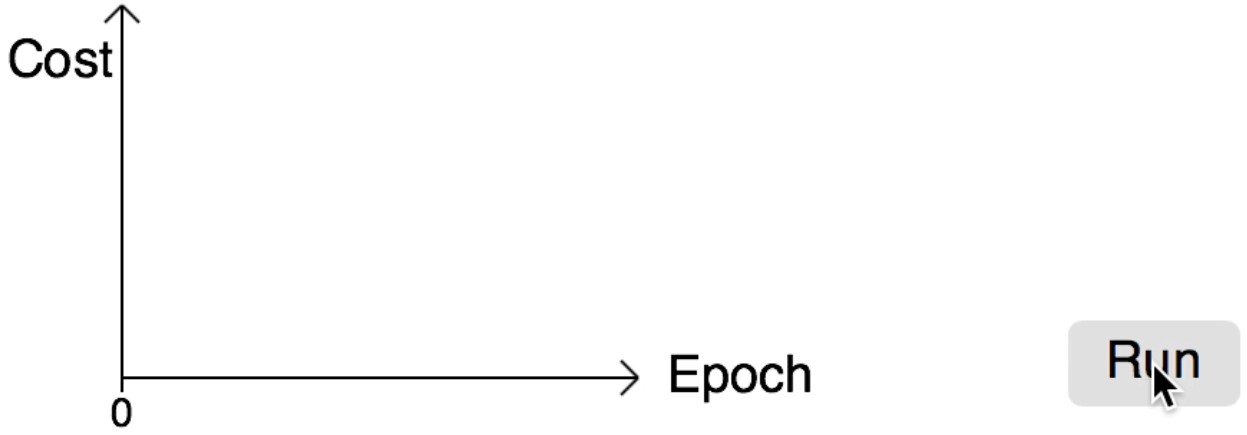
# Practical Implementation of Architectures

- Mean squared error:

$$J(\mathbf{W}) = \sum_{k}^{M} (\mathbf{y}^{(k)} - \mathbf{a}^{(L)})^2$$

least squares objective,
tends to slow training initially



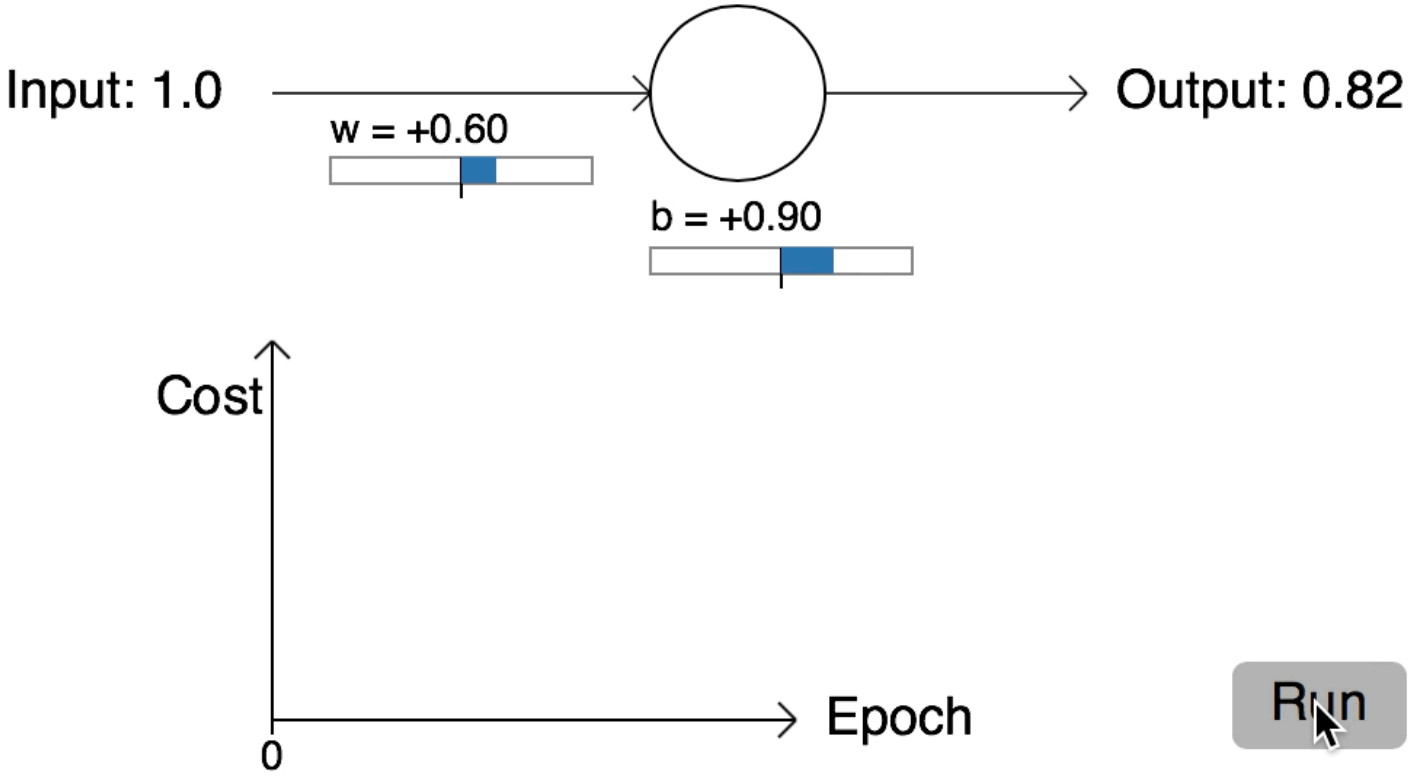

*Neural Networks and Deep Learning*, Michael Nielson, 2015

# Practical Implementation of Architectures

- Our old friend, **Binary Cross entropy**

$$J(\mathbf{W}) = -\left[\mathbf{y}^{(i)} \ln([\mathbf{a}^{(L+1)}]^{(i)}) + (1 - \mathbf{y}^{(i)}) \ln(1 - [\mathbf{a}^{(L+1)}]^{(i)})\right]$$

speeds up initial training



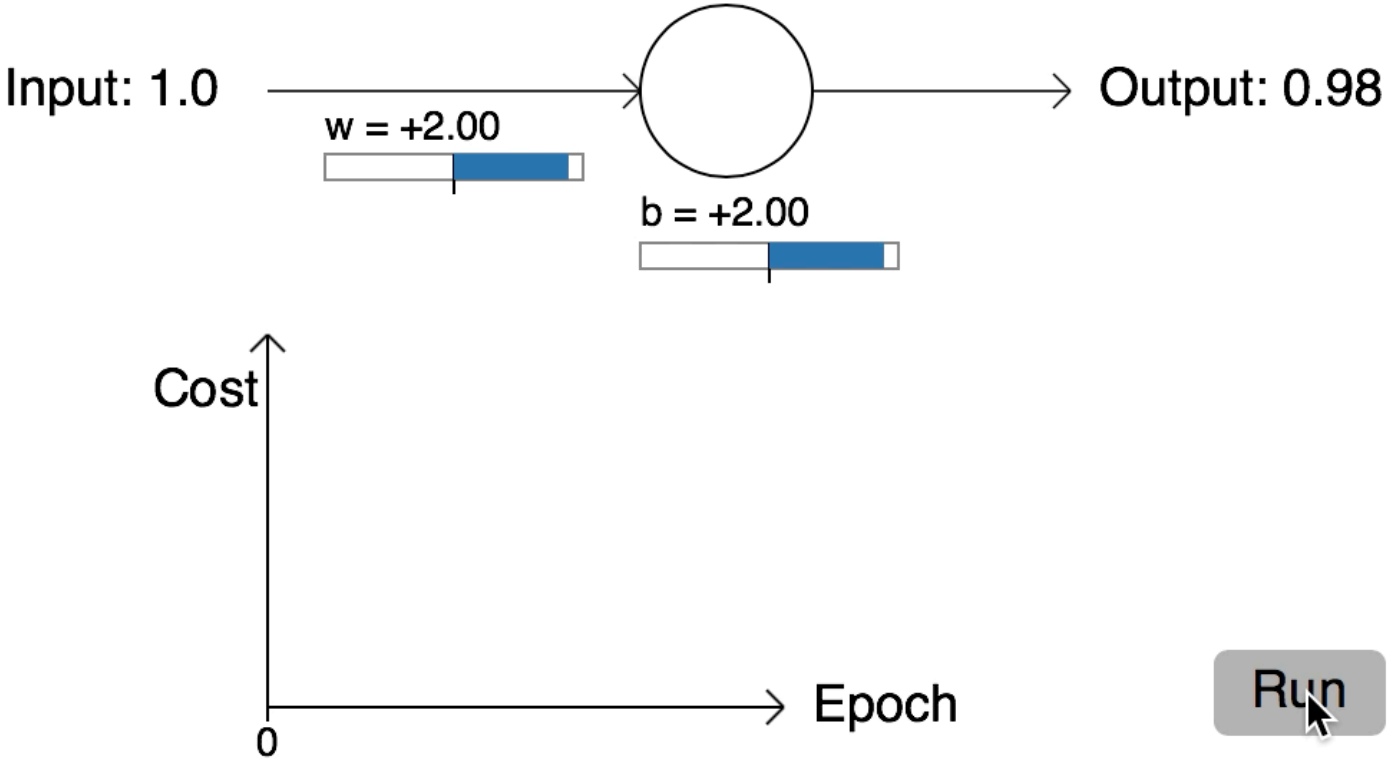

*Neural Networks and Deep Learning*, Michael Nielson, 2015

# Practical Implementation of Architectures

- Negative of MLE: **Binary Cross entropy**

$$J(\mathbf{W}) = -\left[\mathbf{y}^{(i)} \ln([\mathbf{a}^{(L+1)}]^{(i)}) + (1 - \mathbf{y}^{(i)}) \ln(1 - [\mathbf{a}^{(L+1)}]^{(i)})\right]$$

speeds up initial training




*Neural Networks and Deep Learning*, Michael Nielson, 2015

# Practical Implementation of Architectures

$$J(\mathbf{W}) = -\left[\mathbf{y}^{(i)} \ln([\mathbf{a}^{(L+1)}]^{(i)}) + (1-\mathbf{y}^{(i)})\ln(1-[\mathbf{a}^{(L+1)}]^{(i)})\right]$$

likely to speed up initial training

$$\left[\frac{\partial J(\mathbf{W})}{\partial \mathbf{z}^{(L)}}\right]^{(i)} = -\frac{\partial}{\partial \mathbf{z}^{(L)}}\left[\mathbf{y}^{(i)} \ln([\mathbf{a}^{(L+1)}]^{(i)}) + (1-\mathbf{y}^{(i)})\ln(1-[\mathbf{a}^{(L+1)}]^{(i)})\right]$$

only $\mathbf{a}$ has dependence on $\mathbf{z}$

$$= -\left[\mathbf{y}^{(i)}\frac{\partial}{\partial \mathbf{z}^{(L)}}\left(\ln([\mathbf{a}^{(L+1)}]^{(i)})\right) + (1-\mathbf{y}^{(i)})\frac{\partial}{\partial \mathbf{z}^{(L)}}\left(\ln(1-[\mathbf{a}^{(L+1)}]^{(i)})\right)\right]$$

$$= -\left[\mathbf{y}^{(i)}\frac{1}{[\mathbf{a}^{(L+1)}]^{(i)}}\left(\frac{\partial}{\partial \mathbf{z}^{(L)}}[\mathbf{a}^{(L+1)}]^{(i)}\right) + \frac{(1-\mathbf{y}^{(i)})}{1-[\mathbf{a}^{(L+1)}]^{(i)}}\left(-\frac{\partial}{\partial \mathbf{z}^{(L)}}[\mathbf{a}^{(L+1)}]^{(i)}\right)\right]$$

$$= -\left[\mathbf{y}^{(i)}\frac{1}{[\mathbf{a}^{(L+1)}]^{(i)}}\left([\mathbf{a}^{(L+1)}]^{(i)}(1-[\mathbf{a}^{(L+1)}]^{(i)})\right) - \frac{(1-\mathbf{y}^{(i)})}{1-[\mathbf{a}^{(L+1)}]^{(i)}}\left([\mathbf{a}^{(L+1)}]^{(i)}(1-[\mathbf{a}^{(L+1)}]^{(i)})\right)\right]$$

$$= -\left[\mathbf{y}^{(i)}\left(1-[\mathbf{a}^{(L+1)}]^{(i)}\right) - (1-\mathbf{y}^{(i)})\left([\mathbf{a}^{(L+1)}]^{(i)}\right)\right]$$

$$= -\left[\mathbf{y}^{(i)} - \mathbf{y}^{(i)}[\mathbf{a}^{(L+1)}]^{(i)} - [\mathbf{a}^{(L+1)}]^{(i)} + [\mathbf{a}^{(L+1)}]^{(i)}\mathbf{y}^{(i)}\right] \quad = [\mathbf{a}^{(L+1)}]^{(i)} - \mathbf{y}^{(i)}$$

$$\mathbf{V}^{(2)} = -2(\mathbf{Y}-\mathbf{A}^{(3)}) \odot \mathbf{A}^{(3)} \odot (1-\mathbf{A}^{(3)})$$ old update